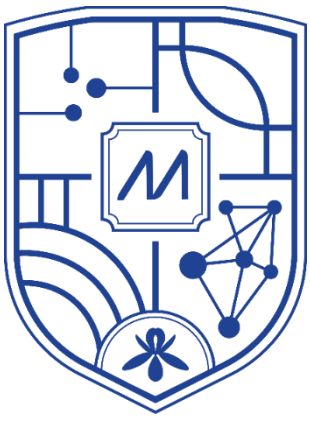
LAPORAN
UJIAN TENGAH SEMESTER (UTS)

IMPLEMENTASI CONTAINER DAN PIPELINE CI/CD PADA APLIKASI ESORIA CODEFEST

Mata Kuliah: Pengembangan dan Operasi
Kelas: IF-B Pagi TA. 2026/2027
Dosen Pengampu: Sunario Megawan, S.Kom., M.Kom.
Kelompok Esoria

Disusun oleh:

Fikri Anwar
231112145
Havidz Andrian
231111899
Muhammad Alfisyahrin
231112853



PROGRAM STUDI S-1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
UNIVERSITAS MIKROSKIL
MEDAN
2026

DAFTAR ISI

BAB I PENDAHULUAN DAN PROFIL APLIKASI 1

1.1 Latar Belakang dan Permasalahan Deployment 1

1.2 Informasi Nama Aplikasi dan Anggota Kelompok 1

1.3 Deskripsi Singkat Aplikasi 2

1.4 Tata Cara Menjalankan dan Menggunakan Aplikasi ESORIA CodeFest 3

1.5 Hasil 4

BAB II SPESIFIKASI TEKNIS DAN SUMBER DAYA APLIKASI 5

2.1 Spesifikasi Teknis dan Arsitektur Aplikasi 5

2.2 Sumber Referensi Source Code dan Struktur Direktori 6

2.3 Informasi File Pendukung dan Aplikasi Tambahan 7

2.4 Hasil 8

BAB III IMPLEMENTASI DAN PENGUJIAN DEPLOYMENT BERBASIS CONTAINER 9

3.1 Pemilihan Engine Container dan Alasan Penggunaan 9

3.2 Perancangan Dockerfile dan Orkestrasi Container 9

3.3 Pengujian Build Image dan Eksekusi Container 11

3.4 Hasil Pengujian dan Analisis Deployment Container 12

3.5 Hasil 13

BAB IV IMPLEMENTASI PIPELINE CI/CD PADA CONTAINER 14

4.1 Arsitektur dan Metode Pengerjaan Pipeline CI/CD 14

4.2 Konfigurasi Script Workflow CI/CD dan Variabel Secrets 15

4.3 Hasil Pengujian Otomatisasi Pipeline CI/CD 20

4.4 Analisis Hasil dan Evaluasi Kecepatan Deployment 21

4.5 Hasil 21

BAB V LAMPIRAN, KONTRIBUSI PENGERJAAN DAN KESIMPULAN 22

5.1 Lampiran UTS ESORIA 22

5.2 Kontribusi Pengerjaan 22

5.3 Kesimpulan 23

DAFTAR PUSTAKA 24

LAMPIRAN A. MATRIKS PEMENUHAN RUBRIK 25

LAMPIRAN B. REKAP PENGUJIAN OTOMATIS 26

LAMPIRAN C. PROSEDUR DAN HASIL VERIFIKASI CONTAINER 27

LAMPIRAN D. PROSEDUR DAN HASIL CI/CD 28

LAMPIRAN E. DOKUMENTASI BUKTI VISUAL SS 01-19 29

BAB I
PENDAHULUAN DAN PROFIL APLIKASI

1.1 Latar Belakang dan Permasalahan Deployment

Pengembangan aplikasi web membutuhkan proses rilis yang cepat, dapat diulang, dan konsisten pada komputer pengembang maupun mesin tujuan. Pada pendekatan manual, perbedaan versi runtime, perubahan konfigurasi, konflik port, dan penyimpanan data yang tidak dipisahkan dari container dapat menyebabkan gangguan setelah pembaruan. Proyek UTS ini menggunakan ESORIA CodeFest, aplikasi pengelolaan kompetisi yang sebelumnya dikembangkan dengan layanan Firebase, sebagai objek adaptasi untuk menunjukkan penggunaan container serta alur Continuous Integration dan Continuous Deployment (CI/CD). Cakupan yang dibahas adalah versi UTS mandiri, bukan seluruh fitur aplikasi Firebase sebelumnya.

Proyek ESORIA CodeFest versi UTS menggabungkan antarmuka web, HTTP REST API Node.js dan penyimpanan SQLite dalam image Docker multi-stage. Docker Compose mengelola port 8088, healthcheck, pembatasan keamanan, serta volume persisten esoria_uts_data. Implementasi telah diuji melalui Docker Desktop dan GitHub Actions pada repository https://github.com/fikriianwarr/esoria-codefest-uts. Workflow Run #2 (27 Oktober 2026, commit 5fe69f9) berhasil menyelesaikan verify, publish ke GHCR, dan deploy melalui runner Ubuntu WSL2. Pada tahap awal, pengujian otomatis melalui GitHub Actions Run #2 berhasil menjalankan 81 dari 81 pengujian tanpa kegagalan. Setelah dilakukan pembaruan dan penyempurnaan kode aplikasi, kelompok melaksanakan pengujian lokal tahap kedua sebagai pengujian versi final dengan hasil 94 dari 94 pengujian berhasil. Pemeriksaan setelah proses Continuous Deployment (CD) juga menunjukkan container dalam kondisi healthy, endpoint /api/health merespons dengan baik, serta identitas rilis pada /api/release sesuai dengan commit pipeline. Bukti pelaksanaan dan pengujian tersebut disertakan pada Lampiran B-E.

1.2 Informasi Nama Aplikasi dan Anggota Kelompok

Objek laporan ini adalah ESORIA CodeFest versi UTS Container & CI/CD (versi 1.0.0) oleh tiga anggota Kelompok Esoria. Dokumentasi demonstrasi disusun dalam satu video kelompok dengan dua pokok bahasan, yaitu container dan CI/CD. Pembagian tanggung jawab berikut menjadi acuan penjelasan setiap anggota, bukan klaim kepemilikan commit; kontribusi aktual harus sesuai kegiatan dan rekaman.

No	Nama Lengkap Mahasiswa	NIM	Bagian Pengerjaan
1	Fikri Anwar	231112145	Container Docker: Dockerfile multi-stage, Compose lokal/deploy, launcher, port, volume, dan healthcheck. Menjelaskan segmen Docker dalam video kelompok.
2	Havidz Andrian	231111899	QA dan dokumentasi: 94 tes, API health/release, persistensi SQLite, bukti screenshot, dan laporan. Menjelaskan pengujian dan verifikasi akhir dalam video yang sama.
3	Muhammad Alfisyahrin	231112853	CI/CD: workflow GitHub Actions, secrets/variables, GHCR, runner WSL2, deployment dan rollback. Menjelaskan segmen CI/CD dalam video kelompok.

1.3 Deskripsi Singkat Aplikasi

ESORIA CodeFest merupakan aplikasi berbasis web yang digunakan untuk mengelola kegiatan kompetisi di bidang teknologi informasi, mulai dari pendaftaran peserta hingga pengelolaan hasil administrasi kompetisi. Pada versi UTS ini, aplikasi menyediakan tiga kategori lomba sebagai data simulasi awal, yaitu Website Programming untuk individu dengan biaya Rp150.000 dan kuota 50 peserta, Cybersecurity Mini CTF untuk individu dengan biaya Rp150.000 dan kuota 50 peserta, serta Full Stack Development untuk tim beranggotakan 2-4 orang dengan biaya Rp500.000 dan kuota 25 tim.

Aplikasi menyediakan fitur pembuatan akun, login, pengelolaan profil, pemilihan kategori lomba, pendaftaran kompetisi, pengisian referensi pembayaran simulasi, pemantauan status pendaftaran, pembatalan pendaftaran yang masih menunggu persetujuan, serta akses tiket peserta setelah pendaftaran disetujui. Peserta juga dapat mengirimkan tautan karya sesuai ketentuan dan tenggat waktu lomba. Sementara itu, administrator bertugas mengelola kategori kompetisi, memeriksa dan memutuskan status pendaftaran, serta memantau statistik dan aktivitas aplikasi. Data pembayaran yang digunakan pada versi ini hanya bersifat simulasi dan tidak melibatkan transaksi keuangan sebenarnya.

Secara teknis, ESORIA CodeFest versi UTS menggunakan Node.js sebagai server aplikasi, SQLite sebagai basis data, serta HTML, CSS, dan JavaScript untuk membangun antarmuka website. Aplikasi dijalankan menggunakan Docker dengan bantuan Docker Compose untuk mengelola container, konfigurasi layanan, dan penyimpanan data. Versi ini dikembangkan secara mandiri untuk kebutuhan implementasi container dan CI/CD sehingga tidak mencakup seluruh fitur ESORIA CodeFest sebelumnya yang menggunakan Firebase.

BUKTI TAMPILAN WEBSITE - TERLAMPIR

SS 01: Beranda; SS 02: direktori kategori; SS 03: dashboard Admin. Kondisi sesudah CD tersedia kembali pada SS 18 dan SS 19. Seluruh bukti visual dimasukkan secara langsung pada Lampiran E.

1.4 Tata Cara Menjalankan dan Menggunakan Aplikasi ESORIA CodeFest

Untuk menjalankan aplikasi ESORIA CodeFest versi UTS pada komputer lokal, pengguna perlu menyiapkan Docker Desktop dan Node.js terlebih dahulu. Aplikasi dijalankan menggunakan container Docker sehingga pengguna tidak perlu menginstal serta mengonfigurasi basis data secara terpisah. Berikut langkah-langkah menjalankan dan menggunakan aplikasi:

1. **Mengunduh file aplikasi**: Unduh file ZIP ESORIA CodeFest UTS melalui tautan OneDrive yang telah disediakan oleh kelompok, kemudian ekstrak seluruh isinya ke dalam folder pada komputer.
2. **Menyiapkan perangkat lunak**: Pastikan Docker Desktop telah terinstal dan berjalan menggunakan Linux Containers, serta Node.js versi 24.22.1 atau versi 24 yang lebih baru telah tersedia. Docker Compose v2 juga harus dapat dijalankan melalui Docker Desktop.
3. **Membuka Command Prompt (CMD)**: Masuk ke folder hasil ekstraksi yang berisi mulai.cmd, kemudian ketik cmd pada kolom alamat File Explorer dan tekan Enter. Command Prompt akan terbuka langsung pada direktori proyek.
4. **Memeriksa lingkungan aplikasi**: Jalankan perintah node --version dan docker compose version pada CMD untuk memastikan Node.js serta Docker Compose dapat dikenali oleh sistem. Pastikan Docker Desktop sudah dalam kondisi berjalan sebelum melanjutkan.
5. **Menjalankan aplikasi**: Ketik perintah mulai.cmd pada CMD, kemudian tekan Enter. Sistem akan memeriksa konfigurasi awal, membuat file .env apabila belum tersedia, membangun Docker image, dan menjalankan container aplikasi secara otomatis. Pada penggunaan pertama, kredensial administrator akan ditampilkan dan perlu disimpan secara pribadi.
6. **Menunggu proses deployment lokal**: Tunggu hingga proses pembangunan image dan pemeriksaan kesehatan container selesai. Setelah berhasil, CMD akan menampilkan alamat website, umumnya http://localhost:8080 atau http://localhost:8088 apabila port utama tidak tersedia. Browser juga akan dibuka secara otomatis menuju aplikasi.

7. **Mengakses dan menggunakan website**: Setelah halaman utama ESORIA CodeFest muncul, pengguna dapat memilih menu kategori lomba, membuat akun melalui halaman pendaftaran, dan masuk menggunakan akun yang telah dibuat. Pengguna kemudian dapat memilih kategori kompetisi, mengisi formulir pendaftaran, serta memantau status pendaftaran melalui dashboard. Tiket peserta dapat dilihat setelah pendaftaran disetujui oleh administrator.
8. **Mengakses halaman administrator**: Administrator dapat masuk menggunakan email dan kata sandi yang tersimpan pada file .env, kemudian membuka halaman /admin untuk memeriksa pendaftaran peserta, menyetujui atau menolak pendaftaran, mengelola kategori kompetisi, serta melihat statistik dan aktivitas aplikasi.
9. **Memeriksa dan menghentikan aplikasi**: Untuk memastikan container berjalan, pengguna dapat mengetik perintah docker compose ps melalui CMD pada direktori proyek. Apabila aplikasi sudah selesai digunakan, container dapat dihentikan menggunakan perintah docker compose down. Perintah tersebut tidak menghapus data SQLite yang tersimpan dalam volume Docker.

Melalui langkah-langkah tersebut, aplikasi ESORIA CodeFest dapat dijalankan secara lokal menggunakan Docker tanpa perlu mengonfigurasi server dan basis data secara manual. Pengguna dapat mengakses fitur pendaftaran dan pengelolaan kompetisi sesuai perannya sebagai peserta maupun administrator. Dengan demikian, penggunaan container mempermudah proses menjalankan aplikasi serta menjaga konsistensi lingkungan sistem.

1.5 Hasil

Website ESORIA versi UTS dijalankan dan diverifikasi pada Docker Desktop dengan port 127.0.0.1:8088; source tersimpan pada GitHub, dan pipeline CI/CD berhasil secara end-to-end pada Run #2. Bab berikutnya menguraikan paket aplikasi, rancangan container, serta bukti pengujian dan implementasi yang dirujuk pada Lampiran E (SS 01-19).

BAB II
SPESIFIKASI TEKNIS DAN SUMBER DAYA APLIKASI

2.1 Spesifikasi Teknis dan Arsitektur Aplikasi

Versi UTS menggunakan arsitektur web monolitik satu origin: browser memuat HTML, CSS, dan JavaScript dari app/public; server HTTP pada app/server.mjs menangani aset serta REST API; app/security.mjs memvalidasi input, autentikasi, token CSRF dan kontrol sesi; app/database.mjs menyediakan SQLite bawaan Node.js (node:sqlite). Port 3000 di dalam container dihubungkan dengan port host yang dipilih launcher. Berkas database esoria.sqlite terletak pada volume Docker sehingga data tetap tersimpan saat container direstart atau dibuat ulang. Image runtime dibangun dari Node.js 24.22.1 Bookworm Slim melalui dua tahap, verify dan runtime, dan menggunakan pengguna non-root.

Komponen Sistem	Teknologi / Spesifikasi	Versi	Fungsi dalam Aplikasi
Bahasa Pemrograman & Runtime	Node.js ES Modules (.mjs); API HTTP native; tanpa dependency npm pihak ketiga	24.22.1; syarat >=24.22.1 <25	Runtime utama eksekusi HTTP server, REST API, kriptografi, test runner, dan akses SQLite bawaan
Arsitektur Web & API	HTML5, CSS3, JavaScript native; Node.js node:http; satu origin	v1.0.0 (Built-in API)	Melayani halaman web dan endpoint API /api/* dengan pembatasan akses dan CSP.
Basis Data (Database)	node:sqlite DatabaseSync; WAL; named volume Docker	SQLite 3 (WAL Mode, Schema v1)	Menyimpan users, sessions, events, registrations dan audit_logs pada /app/data/esoria.sqlite.
Keamanan & Autentikasi	node:crypto (scrypt, token, hashing) serta cookie HttpOnly, SameSite=Strict, CSRF	Built-in Node.js v24.22.1	Autentikasi berbasis peran participant/admin, validasi formulir, perlindungan endpoint dan pembatasan percobaan.
Container Engine & Orkestrasi	Docker Engine/Desktop + Docker Compose v2; image Node.js Bookworm Slim	Dockerfile v1; Compose v2	Multi-stage build, USER node, read_only, cap_drop ALL, tmpfs, healthcheck, volume, pembatasan CPU/RAM.
Port Jaringan & Resolusi Konflik	Container 3000/tcp; host 127.0.0.1:8080 atau alternatif 8088+	TCP / HTTP	Pemilihan otomatis port dan penyimpanan APP_PORT pada .env tanpa mengubah sandi admin.
Kerangka Pengujian Otomatis	Node.js built-in test runner dan container smoke test terpisah	81/81 tes awal; 94/94 tes final lokal	Memverifikasi fungsi aplikasi melalui pengujian otomatis, dengan 81 tes pada GitHub Actions Run #2 dan 94 tes pada pengujian lokal setelah pembaruan kode.

2.2 Sumber Referensi Source Code dan Struktur Direktori

Sumber kode adalah proyek ESORIA CodeFest UTS yang diunggah pada repository GitHub https://github.com/fikriianwarr/esoria-codefest-uts, branch main. Screenshot repository dan berkas .github/workflows/esoria-ci-cd.yml tersedia pada SS 11. Asal pengembangan dari konsep CodeFest terdahulu didokumentasikan pada docs/SUMBER.md, sementara paket UTS berdiri sendiri menggunakan Node.js, REST API, dan SQLite. Workflow dapat diperiksa melalui https://github.com/fikriianwarr/esoria-codefest-uts/blob/main/.github/workflows/esoria-ci-cd.yml; bukti eksekusi Run #2 melalui https://github.com/fikriianwarr/esoria-codefest-uts/actions/runs/40434638852.

Butir Informasi Sumber Kode	Detail dan Tautan (URL) Referensi
Repositori Git Kelompok (Utama)	https://github.com/fikriianwarr/esoria-codefest-uts
Branch Utama Deployment	main; pull_request untuk CI; push pada main; workflow_dispatch. Run #2: https://github.com/fikriianwarr/esoria-codefest-uts/actions/runs/40434638852
Asal Pengembangan Aplikasi	Versi UTS ESORIA dikembangkan dari konsep proyek CodeFest yang semula menggunakan Firebase; beralih ke Node.js dan SQLite dalam container dengan ruang lingkup yang dinyatakan.
Sumber Referensi Kode / Dokumentasi	ZIP akhir berisi README.md, docs/SUMBER.md, docs/CI_CD.md, docs/ANALISIS_RUBRIK.md, source app/, scripts/, tests/ dan workflow .github/. Tidak terdapat folder referensi/firebase-asli, MANIFEST_ASAL.json, atau docs/CATATAN_REVISI.md dalam ZIP akhir.

Struktur Direktori Proyek Aplikasi ESORIA CodeFest (esoria_codefest_uts/)

```
esoria-codefest-uts/
├── mulai.cmd                    # satu launcher Windows
├── .github/workflows/esoria-ci-cd.yml
├── app/                         # server, database, security, admin-repair
│   ├── server.mjs
│   ├── database.mjs
│   ├── security.mjs
│   ├── admin-repair.mjs
│   └── public/                  # HTML, CSS, JavaScript, aset
├── scripts/                     # setup, port, pengujian, deploy, rollback
│   ├── setup.mjs; start-local.mjs; local-port.mjs
│   ├── check.mjs; check-runtime.mjs
│   ├── verify-container.sh; verify-container-local.mjs
│   ├── deploy.sh; rollback.sh; verify-release.mjs
```

Struktur Direktori Proyek Aplikasi ESORIA CodeFest (esoria_codefest_uts/)

```
│   └── deploy-config.mjs; deployment-common.sh; lainnya
├── tests/                          # 11  berkas pengujian; 81 tes pada GitHub
Actions Run #2
├── docs/                           # ANALISIS_RUBRIK, CI_CD, SUMBER
├── Dockerfile
├── compose.yaml; compose.deploy.yaml
├── .dockerignore; .gitignore; .env.example
├── package.json; package-lock.json
└── README.md
Catatan: konfigurasi rahasia .env dibuat lokal, tidak ikut ZIP.
```

Direktori dalam ZIP mengikuti pemisahan tanggung jawab: app untuk aplikasi, scripts untuk pengoperasian serta deployment, tests untuk pengujian, .github/workflows untuk pipeline, dan docs untuk dokumentasi teknis. Satu-satunya launcher Windows yang ditujukan kepada pengguna adalah mulai.cmd; skrip Node.js dan Bash tetap diperlukan sebagai bagian sistem build dan CI/CD. Susunan aktual disederhanakan tanpa menyertakan arsip Firebase awal, keluaran uji historis, atau laporan Word di dalam ZIP aplikasi.

2.3 Informasi File Pendukung dan Aplikasi Tambahan

File pendukung mempertahankan kemampuan aplikasi dijalankan dan diuji melalui container. Dockerfile menerapkan tahap verifikasi sebelum image runtime, Compose mengelola layanan web dan volume, sementara .env dibuat lokal agar sandi admin tidak bocor ke source. Skrip verify-container.sh serta verify-container-local.mjs dipakai untuk membuktikan kesehatan, smoke test, dan persistensi pada volume pengujian terpisah. Skrip deploy.sh, verify-release.mjs dan rollback.sh digunakan pada pipeline CD dengan image digest, sedangkan docs/CI_CD.md mendokumentasikan cara mengaktifkan runner, secrets dan variables.

Nama File Pendukung / Aplikasi Tambahan	Kategori	Fungsi dan Peran dalam Mendukung Aplikasi
Dockerfile	File Pendukung Container	Mendefinisikan multi-stage build: stage verify (menjalankan npm ci, npm run check, dan npm test) serta stage runtime berbasis node:24.22.1-bookworm-slim dengan user non-root (node) dan HEALTHCHECK.
compose.yaml & compose.deploy.yaml	File Orkestrasi Container	Mengatur layanan container web, pemetaan port host/container, named volume esoria_uts_data, read-only rootfs, tmpfs /tmp, cap_drop: [ALL], no-new-privileges, serta batas memori 256m & CPU 1.0.

Nama File Pendukung / Aplikasi Tambahan	Kategori	Fungsi dan Peran dalam Mendukung Aplikasi
.dockerignore & .env (.env.example)	File Optimasi & Konfigurasi	Mengabaikan file privat dan keluaran lokal; .env.example hanya contoh. Berkas compose.yaml dan compose.deploy.yaml wajib tersedia pada konteks build untuk tes regresi.
package.json & package-lock.json	File Manifes & Script	Menyediakan perintah npm run check, npm test, start:docker, check:docker, verify:container, serta verify:container:windows untuk pemeriksaan aplikasi dan pengujian container.
mulai.cmd, setup.mjs, local-port.mjs, start-local.mjs	Script Launcher & Resolusi Port	Launcher tunggal memeriksa Node/Docker; setup hanya membuat .env jika belum ada; port 8080 fallback ke 8088+; memulai Compose dan membuka browser.
check.mjs, verify-container.sh, verify-container-local.mjs, deploy.sh, verify-release.mjs, rollback.sh	Script Verifikasi, CD & Rollback	Mendukung pemeriksaan sintaks, pengujian aplikasi dan container, smoke test, persistensi volume, deployment image berbasis digest, verifikasi commit dan kesehatan layanan, serta mekanisme rollback.
SQLite (node:sqlite) & Volume esoria_uts_data	Layanan Basis Data Persisten	Menyimpan basis data relasional /app/data/esoria.sqlite dalam mode jurnal WAL pada named volume Docker agar seluruh transaksi pendaftaran, sesi, dan log audit bertahan saat container diganti.
GitHub Actions, GHCR (ghcr.io), & Self-Hosted Runner Linux x64	Aplikasi & Layanan CI/CD	Workflow verify selalu berjalan; publish dan deploy membutuhkan ENABLE_CD=true, kredensial admin, port tujuan, serta runner berlabel esoria-uts.

2.4 Hasil

Proyek UTS ESORIA CodeFest berhasil menyediakan aplikasi mandiri dengan Dockerfile multi-stage, dua konfigurasi Docker Compose, launcher Windows mulai.cmd, skrip deployment, serta dokumentasi teknis. Pengujian awal melalui GitHub Actions Run #2 menghasilkan 81 pengujian berhasil tanpa kegagalan. Setelah pembaruan dan penyempurnaan kode, pengujian lokal versi final mencatat 94 pengujian berhasil tanpa kegagalan. Selain itu, image Docker berhasil dibangun, persistensi volume telah diuji, dan pipeline CI/CD berhasil menjalankan tahapan verify, publish, serta deploy. Seluruh hasil pengujian dan bukti implementasi didokumentasikan pada Lampiran B-E.

BAB III
IMPLEMENTASI DAN PENGUJIAN DEPLOYMENT BERBASIS CONTAINER

3.1 Pemilihan Engine Container dan Alasan Penggunaan

Docker Engine/Docker Desktop dengan Docker Compose v2 dipilih sebagai platform containerisasi karena dapat mengemas lingkungan Node.js aplikasi bersama source dan konfigurasi runtime, mengurangi perbedaan antara komputer pengembang dan mesin deployment. Dockerfile dalam ZIP terdiri atas tahap verify yang menjalankan pemeriksaan serta pengujian, kemudian tahap runtime yang menyalin hanya source aplikasi yang dibutuhkan. Node.js versi 24.22.1 Bookworm Slim menjadi base image, dengan proses dijalankan sebagai USER node (non-root). Pilihan ini cocok untuk studi kasus deployment cepat ESORIA yang memiliki HTTP API, frontend statis, dan database SQLite.

Berkas Compose menerapkan read_only: true, tmpfs /tmp, cap_drop: [ALL], no-new-privileges, mem_limit 256m, cpus 1.0, pids_limit 100, serta healthcheck dan restart policy unless-stopped. Volume esoria_uts_data dipasang ke /app/data. Pada pemeriksaan aktual, service esoria-uts-web-1 menjalankan image rilis pada 127.0.0.1:8088->3000/tcp dengan status Up (healthy), dan endpoint /api/health menunjukkan database ok (SS 16-17).

3.2 Perancangan Dockerfile dan Orkestrasi Container

Dockerfile menyalin package.json dan lockfile, menjalankan npm ci, menyalin source aplikasi, skrip pengujian, dua berkas Compose, dan workflow GitHub Actions untuk mendukung pengujian regresi, kemudian menjalankan npm run check && npm test sebelum image runtime dibentuk. Perbaikan penting dari revisi sebelumnya adalah memastikan compose.yaml tidak dikecualikan oleh .dockerignore. Pada tahap runtime, aplikasi berjalan di port internal 3000 dengan healthcheck /api/health dan non-root user. Konfigurasi Compose lokal membangun image esoria-uts:local, sedangkan Compose deployment memasang image GHCR yang merujuk pada digest hasil pipeline.

Konfigurasi Berkas Dockerfile Multi-Stage Aplikasi ESORIA CodeFest (Dockerfile)

```
# syntax=docker/dockerfile:1
FROM node:24.22.1-bookworm-slim AS verify
WORKDIR /build
COPY package.json package-lock.json ./
RUN npm ci --ignore-scripts --no-audit --no-fund
COPY app ./app
COPY scripts ./scripts
COPY tests ./tests
# Dipakai oleh tests/deploy-regression.test.mjs saat verifikasi di image
Docker.
COPY compose.deploy.yaml ./compose.deploy.yaml
COPY compose.yaml ./compose.yaml
COPY .github/workflows/esoria-ci-cd.yml ./.github/workflows/esoria-ci-cd.yml
RUN npm run check && npm test

FROM node:24.22.1-bookworm-slim AS runtime
ARG RELEASE_SHA=local
LABEL org.opencontainers.image.title="ESORIA CodeFest UTS" \
      org.opencontainers.image.description="Container aplikasi kompetisi
ESORIA untuk UTS" \
      org.opencontainers.image.revision="${RELEASE_SHA}"
ENV NODE_ENV=production \
    PORT=3000 \
    DATA_DIR=/app/data \
    RELEASE_SHA=${RELEASE_SHA}
WORKDIR /app
COPY --from=verify --chown=node:node /build/app ./app
RUN mkdir -p /app/data && chown node:node /app/data
USER node
EXPOSE 3000
HEALTHCHECK --interval=10s --timeout=3s --start-period=15s --retries=6 \
  CMD node -e
"fetch('http://127.0.0.1:3000/api/health').then(r=>{if(!r.ok)process.exit(1)}
).catch(()=>process.exit(1))"
CMD ["node", "app/server.mjs"]
```

Compose lokal membangun image esoria-uts:local; Compose deployment memakai referensi image GHCR berbasis digest dari ESORIA_IMAGE tanpa membangun ulang di server. Keduanya memetakan port 3000, menggunakan volume esoria_uts_data, dan menerapkan pembatasan keamanan. Variabel ADMIN_SYNC_ON_START=true menjaga akun admin sesuai .env ketika layanan dimulai. Potongan Compose berikut merupakan ringkasan; konfigurasi lengkap terdapat di ZIP proyek.

Konfigurasi Orkestrasi Container Lokal (compose.yaml) dan Produksi CD (compose.deploy.yaml)

```
# compose.yaml (inti)
name: esoria-uts
services:
  web:
    image: esoria-uts:local
    build:
      context: .
      args: { RELEASE_SHA: '${RELEASE_SHA:-local}' }
    ports:
      - '${BIND_ADDRESS:-127.0.0.1}:${APP_PORT:-8080}:3000'
    environment:
      ADMIN_EMAIL: ${ADMIN_EMAIL:?Konfigurasi admin diperlukan}
      ADMIN_PASSWORD: ${ADMIN_PASSWORD:?Konfigurasi admin diperlukan}
      ADMIN_SYNC_ON_START: 'true'
      COOKIE_SECURE: ${COOKIE_SECURE:-false}
      APP_NAME: ${APP_NAME:-ESORIA CodeFest}
    volumes: [esoria-data:/app/data]
    restart: unless-stopped
    init: true
    read_only: true
    tmpfs: ['/tmp:size=16m,mode=1777']
    cap_drop: [ALL]
    security_opt: [no-new-privileges:true]
    mem_limit: 256m
    cpus: 1.0
    pids_limit: 100
volumes:
  esoria-data:
    name: esoria_uts_data

# compose.deploy.yaml: properti setara, tetapi service.web.image
# menggunakan ${ESORIA_IMAGE:?Image digest hasil CI diperlukan}
# dan TIDAK memuat build: agar image hasil CI tetap identik.
```

3.3 Pengujian Build Image dan Eksekusi Container

Pengujian aplikasi dan container dilakukan menggunakan npm run check, npm test, Docker Desktop, dan skrip verifikasi container. Pada tahap awal, pengujian yang tercatat dalam GitHub Actions Run #2 menghasilkan 81 dari 81 tes lulus tanpa kegagalan. Setelah pembaruan dan penyempurnaan kode aplikasi, pengujian lokal tahap kedua menghasilkan 94 dari 94 tes lulus, terdiri atas 85 pengujian Windows natif dan 9 pengujian Linux terisolasi. Selain itu, proses build Docker, smoke test, restart, recreate, dan pemeriksaan persistensi volume berhasil dilakukan. Pengujian setelah deployment juga menunjukkan bahwa container berjalan sehat, API dan database merespons, serta identitas rilis sesuai dengan commit pipeline yang digunakan.

Perintah CLI Pembuatan Image, Resolusi Port Otomatis, dan Pengujian Container (Docker)

```
# WINDOWS: jalankan dari folder proyek setelah Docker Desktop aktif
# 1. Mulai website melalui launcher utama:
mulai.cmd

# 2. Bukti container dan konfigurasi runtime:
docker compose ps
docker compose port web 3000
docker compose logs --tail 60 web
npm run check:docker

# 3. Pengujian container end-to-end terisolasi:
npm run verify:container:windows

# 4. Pemeriksaan source tanpa Docker:
npm run check
npm test

# 5. Menghentikan container tanpa menghapus database:
docker compose down
# Jangan gunakan docker compose down -v kecuali reset data disengaja.
```

3.4 Hasil Pengujian dan Analisis Deployment Container

Hasil eksekusi aktual memperlihatkan image Docker berhasil dibangun, container esoria-uts-web-1 berstatus Up (healthy), port 127.0.0.1:8088 dipetakan ke 3000/tcp, dan runtime menggunakan user node non-root. Pemeriksaan /api/health menghasilkan status=ok dan database=ok. Audit lokal mencatat keberhasilan restart/recreate dan persistensi volume pada lingkungan pengujian, sedangkan deployment aktual kembali menggunakan volume bernama esoria_uts_data. Sebelum aktivasi CD dibuat pula snapshot SQLite menggunakan Backup API, dengan integrity_check=ok dan SHA256 salinan cocok (SS 13). Keberadaan volume saja tidak membuktikan seluruh data peserta historis utuh; saat screenshot admin diambil, statistik masih nol (SS 19).

Parameter Pengujian Container	Hasil Eksekusi	Status Validasi
Nama & Tag Container Image	esoria-uts:local (lokal); esoria-uts:ci (CI); GHCR @sha256 untuk CD	TERVERIFIKASI - build Docker dan publish GHCR berhasil; SS 04, SS 14
Base Image & Isolasi Keamanan	USER node, read_only, cap_drop ALL, RAM 256m, CPU 1.0	TERVERIFIKASI - non-root user dan pembatasan runtime; SS 08

Parameter Pengujian Container	Hasil Eksekusi	Status Validasi
Gerbang Kualitas & Waktu Uji Build	npm run check: LULUS; pengujian CI/CD Run #2: 81/81 lulus, 0 gagal; pengujian lokal final: 94/94 lulus, 0 gagal.	TERVERIFIKASI - pengujian awal pada Gambar E.09 dan pengujian lokal final pada Gambar E.10.
Pemetaan Port (Port Forwarding)	Port aktual 127.0.0.1:8088 -> 3000/tcp	TERVERIFIKASI - SS 05-07; SS 16
Status Kesehatan & Persistensi (docker ps)	Status healthy; API status/database ok; audit restart/recreate lulus pada volume uji; volume utama esoria_uts_data tetap tersedia	TERVERIFIKASI - SS 05, SS 07, SS 10, SS 16-17

Bukti pelaksanaan ditempatkan pada Lampiran E: SS 04-10 untuk build, keamanan, port, API, tes otomatis, serta persistensi; SS 13 untuk snapshot SQLite; dan SS 16-17 untuk pemeriksaan container setelah deployment. Semua output asli menunjukkan verifikasi runtime, bukan sekadar rancangan yang belum dijalankan.

BUKTI DOCKER DAN PERSISTENSI - SUDAH TERLAMPIR

SS 04: build Docker berhasil; SS 05: container healthy; SS 06: port dan log; SS 07: Healthcheck/Release lokal; SS 08: user non-root dan volume; SS 09: 81 tes lulus; SS 10: persistensi restart/recreate; SS 16-17: kesehatan service dan volume setelah Continuous Deployment. Snapshot SQLite sebelum CD tersedia pada SS 13. Tidak ada password/token yang ditampilkan.

3.5 Hasil

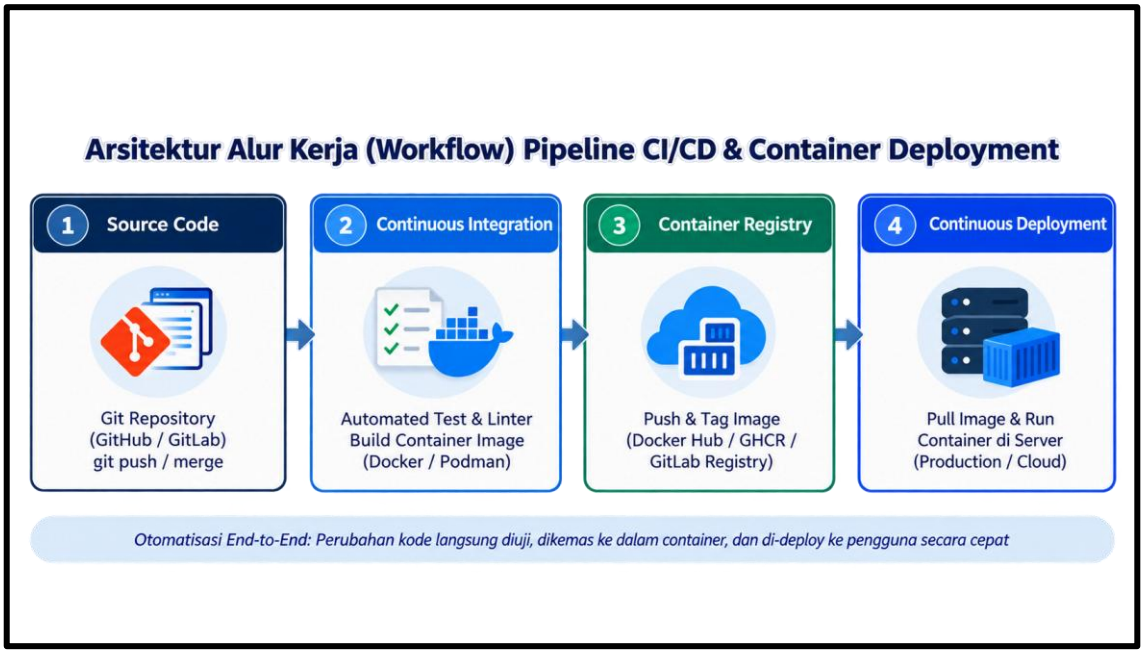
Implementasi container ESORIA CodeFest berhasil dijalankan pada komputer Windows menggunakan Docker Desktop. Proses build multi-stage, penggunaan non-root user, healthcheck, pemetaan port 8088, serta penyimpanan SQLite melalui volume Docker telah diuji. Pengujian awal menghasilkan 81 tes lulus, sedangkan pengujian lokal versi final setelah pembaruan kode menghasilkan 94 tes lulus tanpa kegagalan. Hasil tersebut menunjukkan bahwa pengujian aplikasi telah diperluas pada tahap akhir, sementara pemeriksaan container dan persistensi data juga berhasil didokumentasikan. Verifikasi terhadap keutuhan data historis tetap memerlukan pemeriksaan isi database secara terpisah.

BAB IV
IMPLEMENTASI PIPELINE CI/CD PADA CONTAINER

4.1 Arsitektur dan Metode Pengerjaan Pipeline CI/CD

Pipeline ESORIA menggunakan GitHub Actions dan GitHub Container Registry (GHCR) dengan workflow https://github.com/fikriianwarr/esoria-codefest-uts/blob/main/.github/workflows/esoria-ci-cd.yml. Tiga job utama adalah verify, publish, dan deploy. Trigger meliputi pull_request, push pada main, dan workflow_dispatch. Job verify dapat dijalankan tanpa CD, sedangkan publish dan deploy dibatasi pada branch main dengan repository variable ENABLE_CD=true. Run #2 yang dipicu secara manual (workflow_dispatch) pada commit 5fe69f9 membuktikan ketiga job selesai dengan status Success; pemicu push baru belum diuji sebagai eksperimen terpisah.

Job verify memasang Node.js 24.22.1, menginstal dependency dari lockfile, memeriksa source, menjalankan 81 tes, dan membangun serta menguji image container. Job publish mengirim image yang telah diuji ke GHCR dan menghasilkan referensi berbasis digest. Job deploy menggunakan runner Linux x64 esoria-wsl-uts pada Ubuntu WSL2 dengan label esoria-uts untuk memasang image, memeriksa kesehatan API/database, mencocokkan commit, dan menyiapkan mekanisme pemulihan apabila rilis gagal. Hasil Run #2: verify 1 menit 13 detik, publish 29 detik, deploy 1 menit 21 detik, total durasi workflow 9 menit 48 detik (termasuk antrean dan overhead).



Gambar Alur kerja otomatis Continuous Integration (CI) dan Continuous Deployment (CD) berbasis container.

4.2 Konfigurasi Script Workflow CI/CD dan Variabel Secrets

Berkas .github/workflows/esoria-ci-cd.yml yang ditampilkan berikut adalah konfigurasi yang diunggah pada repository GitHub. Pada pelaksanaan UTS, dua Repository Secrets (ESORIA_ADMIN_EMAIL dan ESORIA_ADMIN_PASSWORD) disimpan tanpa mengungkap nilai; Repository Variables ENABLE_CD=true, ESORIA_APP_PORT=8088, ESORIA_BIND_ADDRESS=127.0.0.1, dan ESORIA_COOKIE_SECURE=false dipakai untuk HTTP lokal. Pipeline berhasil dijalankan dari main melalui workflow_dispatch; bukti sukses nyata ada pada SS 14.

Source proyek telah diunggah pada https://github.com/fikriianwarr/esoria-codefest-uts dengan workflow di .github/workflows/esoria-ci-cd.yml. Pengaturan GitHub Actions menggunakan secrets untuk email dan password admin, serta variables untuk pengaktifan CD, port, bind, dan secure cookie. Token GITHUB_TOKEN berasal dari GitHub Actions dan tidak perlu ditulis sebagai secret pribadi. Nilai rahasia tidak disertakan pada laporan maupun screenshot.

Mesin deployment memakai Ubuntu 24.04 WSL2 yang terintegrasi dengan Docker Desktop. Self-hosted runner bernama esoria-wsl-uts menggunakan label self-hosted, Linux, X64, dan esoria-uts. Runner sempat Offline ketika proses lokal tidak berjalan, kemudian terkoneksi kembali dan mengambil job Deploy. Setelah deployment selesai, statusnya Idle (SS 12 dan SS 15). Konfigurasi menggunakan host 127.0.0.1 dan port 8088.

Pembuktian end-to-end didapat pada GitHub Actions Run #2 (https://github.com/fikriianwarr/esoria-codefest-uts/actions/runs/40434638852) dengan tiga job sukses, artefak tested-container, dan image GHCR berbasis digest sha256:98fc41ff98001539068478aadddc9bdb5661e5e03125fbfa66bf425199d8bfd9. Versi aplikasi 1.0.0 dan commit 5fe69f90675504bcfc599efebd9fd5de04e5bc98 dikembalikan /api/release sesudah deployment (SS 17). Run ini dipicu manual pada main; demonstrasi perubahan antarmuka otomatis setelah push yang berbeda belum didokumentasikan. Pengujian rollback bersumber pada tes regresi, bukan bukti insiden rollback produksi.

KONFIGURASI WORKFLOW GITHUB ACTIONS (YAML)

```
name: ESORIA UTS - CI and CD
on:
  pull_request:
  push:
    branches: [main]
```

KONFIGURASI WORKFLOW GITHUB ACTIONS (YAML)

```
  workflow_dispatch:
permissions:
  contents: read
concurrency:
  group: esoria-${{ github.ref }}
  cancel-in-progress: false
jobs:
  verify:
    name: Test application and container
    runs-on: ubuntu-latest
    timeout-minutes: 20
    steps:
      - uses: actions/checkout@v6
        with:
          persist-credentials: false
      - uses: actions/setup-node@v6
        with:
          node-version: '24.22.1'
          cache: npm
      - name: Check required CD settings when deployment is enabled
        if: github.event_name != 'pull_request' && github.ref ==
'refs/heads/main' && vars.ENABLE_CD == 'true'
        env:
          ENABLE_CD: ${{ vars.ENABLE_CD }}
          ESORIA_ADMIN_EMAIL: ${{ secrets.ESORIA_ADMIN_EMAIL }}
          ESORIA_ADMIN_PASSWORD: ${{ secrets.ESORIA_ADMIN_PASSWORD }}
          ESORIA_APP_PORT: ${{ vars.ESORIA_APP_PORT }}
          ESORIA_BIND_ADDRESS: ${{ vars.ESORIA_BIND_ADDRESS }}
          ESORIA_COOKIE_SECURE: ${{ vars.ESORIA_COOKIE_SECURE }}
        run: node scripts/check-cd-config.mjs
      - name: Install from lockfile
        run: npm ci --ignore-scripts --no-audit --no-fund
      - name: Syntax and local assets
        run: npm run check
      - name: API, authorization, business rules and persistence
        run: npm test
      - name: Build and exercise actual container
```

KONFIGURASI WORKFLOW GITHUB ACTIONS (YAML)

```
        env:
          CI_IMAGE: esoria-uts:ci
        run: bash scripts/verify-container.sh
      - name: Save tested image
        if: github.event_name != 'pull_request' && vars.ENABLE_CD == 'true'
        shell: bash
        run: |
          set -Eeuo pipefail
          docker save esoria-uts:ci | gzip > "$RUNNER_TEMP/esoria-
image.tar.gz"
      - uses: actions/upload-artifact@v4
        if: github.event_name != 'pull_request' && vars.ENABLE_CD == 'true'
        with:
          name: tested-container
          path: ${{ runner.temp }}/esoria-image.tar.gz
          retention-days: 2
  publish:
    name: Publish tested image to GHCR
    needs: verify
    if: github.event_name != 'pull_request' && github.ref ==
'refs/heads/main' && vars.ENABLE_CD == 'true'
    runs-on: ubuntu-latest
    timeout-minutes: 10
    permissions:
      contents: read
      packages: write
    outputs:
      image: ${{ steps.push.outputs.image }}
    steps:
      - uses: actions/download-artifact@v4
        with:
          name: tested-container
          path: ${{ runner.temp }}/image
      - name: Push exact image that passed tests
        id: push
        env:
          GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}
```

KONFIGURASI WORKFLOW GITHUB ACTIONS (YAML)

```
          GH_ACTOR: ${{ github.actor }}
          REPOSITORY: ${{ github.repository }}
          COMMIT_SHA: ${{ github.sha }}
        shell: bash
        run: |
          set -Eeuo pipefail
          export DOCKER_CONFIG="$RUNNER_TEMP/docker-auth"
          trap 'rm -rf "$DOCKER_CONFIG"' EXIT
          printf '%s' "$GH_TOKEN" | docker login ghcr.io -u "$GH_ACTOR" --
password-stdin
          gunzip -c "$RUNNER_TEMP/image/esoria-image.tar.gz" | docker load
          image="ghcr.io/${REPOSITORY,,}/esoria-uts"
          docker tag esoria-uts:ci "$image:sha-$COMMIT_SHA"
          docker push "$image:sha-$COMMIT_SHA"
          digest=$(docker inspect --format '{{index .RepoDigests 0}}'
"$image:sha-$COMMIT_SHA")
          printf 'image=%s\n' "$digest" >> "$GITHUB_OUTPUT"
          printf '### Tested container\n`%s`\n' "$digest" >>
"$GITHUB_STEP_SUMMARY"
  deploy:
    name: Deploy on group machine
    needs: publish
    if: vars.ENABLE_CD == 'true' && github.ref == 'refs/heads/main'
    runs-on: [self-hosted, Linux, X64, esoria-uts]
    timeout-minutes: 15
    permissions:
      contents: read
      packages: read
    steps:
      - uses: actions/checkout@v6
        with:
          persist-credentials: false
      - uses: actions/setup-node@v6
        with:
          node-version: '24.22.1'
      - name: Deploy, check health, match commit and rollback on failure
        env:
```

KONFIGURASI WORKFLOW GITHUB ACTIONS (YAML)

```
          GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}
          GH_ACTOR: ${{ github.actor }}
          ESORIA_IMAGE: ${{ needs.publish.outputs.image }}
          ESORIA_EXPECTED_SHA: ${{ github.sha }}
          ESORIA_ADMIN_EMAIL: ${{ secrets.ESORIA_ADMIN_EMAIL }}
          ESORIA_ADMIN_PASSWORD: ${{ secrets.ESORIA_ADMIN_PASSWORD }}
          ESORIA_APP_PORT: ${{ vars.ESORIA_APP_PORT }}
          ESORIA_BIND_ADDRESS: ${{ vars.ESORIA_BIND_ADDRESS }}
          ESORIA_COOKIE_SECURE: ${{ vars.ESORIA_COOKIE_SECURE }}
        shell: bash
        run: |
          set -Eeuo pipefail
          export DOCKER_CONFIG="$RUNNER_TEMP/docker-auth-esoria"
          trap 'rm -rf "$DOCKER_CONFIG"' EXIT
          printf '%s' "$GH_TOKEN" | docker login ghcr.io -u "$GH_ACTOR" --
password-stdin
         bash scripts/deploy.sh
          printf '### Deployment completed\nCommit `%s` is running and healthy.\n'
"$ESORIA_EXPECTED_SHA" >> "$GITHUB_STEP_SUMMARY"
```

Rahasia dan parameter deployment tidak disimpan di repository. GitHub Actions menggunakan secret ESORIA_ADMIN_EMAIL serta ESORIA_ADMIN_PASSWORD, token GITHUB_TOKEN yang disediakan otomatis oleh GitHub, dan variables ENABLE_CD, ESORIA_APP_PORT, ESORIA_BIND_ADDRESS serta ESORIA_COOKIE_SECURE. Nilai ESORIA_APP_PORT wajib sesuai port kosong pada mesin runner (terutama jika 8080 sudah dipakai). Ketika ENABLE_CD=true, skrip check-cd-config.mjs memeriksa kelengkapan dan validitas konfigurasi sebelum melanjutkan. Password, token, dan isi .env tidak boleh ditampilkan pada screenshot atau laporan.

Nama Secret / Variabel	Lokasi Konfigurasi	Kegunaan dalam Pipeline Otomatis
GITHUB_TOKEN	Secret bawaan GitHub Actions	Autentikasi GHCR; publish packages:write, deploy packages:read.
ESORIA_ADMIN_EMAIL	Repository Secrets	Email akun admin pada mesin deployment.
ESORIA_ADMIN_PASSWORD	Repository Secrets	Sandi admin kuat; jangan ditulis di laporan.

Nama Secret / Variabel	Lokasi Konfigurasi	Kegunaan dalam Pipeline Otomatis
ENABLE_CD	Repository Variables	Aktif: true; Run #2 mencakup verify, publish, deploy
ESORIA_APP_PORT	Repository Variables	8088, dibuktikan oleh pemetaan port 127.0.0.1:8088 -> 3000
ESORIA_BIND_ADDRESS / ESORIA_COOKIE_SECURE	Repository Variables	127.0.0.1 dan false untuk HTTP lokal; dibuktikan melalui pengaturan repository

4.3 Hasil Pengujian Otomatisasi Pipeline CI/CD

Bukti nyata tersedia untuk GitHub Actions Run #2: https://github.com/fikriianwarr/esoria-codefest-uts/actions/runs/40434638852. Status keseluruhan Success, pada commit 5fe69f9, dengan verify (1m 13s), publish (29s), dan deploy (1m 21s) semuanya hijau. Total durasi tertera 9m 48s, termasuk waktu antrean. Job publish menghasilkan image GHCR @sha256 yang dicatat di Lampiran D. SS 14 menunjukkan rangkaian pekerjaan selesai; SS 15 menunjukkan runner kembali Idle. Pemeriksaan runtime sesudah deploy menghasilkan status=ok, database=ok, container healthy, dan SHA rilis yang cocok (SS 16-17).

Hasil 81 pengujian tersebut merupakan catatan pengujian yang dijalankan melalui GitHub Actions Run #2. Setelah proses tersebut, kelompok melakukan pengujian lokal lanjutan terhadap kode aplikasi yang telah diperbarui dan memperoleh hasil 94 pengujian berhasil. Hasil pengujian final tersebut didokumentasikan secara terpisah pada Lampiran B dan E, sehingga tidak dianggap sebagai keluaran dari GitHub Actions Run #2.

BUKTI GITHUB ACTIONS - WORKFLOW #2 BERHASIL	
Repository	https://github.com/fikriianwarr/esoria-codefest-uts
File workflow	Buka esoria-ci-cd.yml pada GitHub
Run sukses	Run #2 - Success (27 Oktober 2026)
Commit terpasang	5fe69f90675504bcfc599efebd9fd5de04e5bc98
Digest image GHCR	ghcr.io/fikriianwarr/esoria-codefest-uts/esoria-uts@ sha256:98fc41ff98001539068478aadddc9bdb5661e5e03125fbfa66bf425199d8bfd9
Hasil pipeline	Verify, Publish, dan Deploy: Success. Endpoint /api/release menunjukkan SHA yang sesuai; bukti SS 11-17.

BUKTI GITHUB ACTIONS - WORKFLOW #2 BERHASIL	
Batas pengujian	Run #2 dipicu manual (workflow_dispatch). Tidak diklaim sebagai hasil pengujian push baru.
Akses repository	Private saat bukti dibuat; dosen perlu diberi akses jika diminta memeriksa kode.

4.4 Analisis Hasil dan Evaluasi Kecepatan Deployment

Pipeline berhasil memisahkan quality gate dari penerbitan image, memakai image teruji berbasis digest, serta memvalidasi kesehatan layanan dan identitas commit sesudah pemasangan. GitHub Actions Run #2 mencatat durasi total 9m 48s; verify 1m 13s; publish 29s; deploy 1m 21s. Selisih antara total dan penjumlahan durasi job mencerminkan waktu antrean dan overhead lain yang terlihat pada proses: deploy sempat menunggu runner Offline sampai proses ./run.sh diaktifkan kembali. Data tersebut adalah durasi satu run, bukan rata-rata dari beberapa eksekusi, dan tidak dapat dipakai sebagai klaim percepatan terhadap deployment manual tanpa pengukuran pembanding.

Tahapan Proses dalam Pipeline CI/CD	Durasi Run #2 (bukan rata-rata)	Status Otomatisasi
verify: setup Node, npm ci, check, 81 tes	1m 13s (total job verify)	SUCCESS - 81 tes dan container verify
verify: build container, smoke, restart/recreate, artifact	Tergabung dalam job verify 1m 13s	SUCCESS - build, smoke, persistence, artifact
publish: memuat artifact teruji dan push digest GHCR	29 detik	SUCCESS - GHCR image @sha256
deploy: pull digest, Compose, health/SHA, rollback	1m 21s	SUCCESS - runner Ubuntu; health dan SHA cocok
Total deploy end-to-end	9m 48s total workflow (termasuk antrean)	SUCCESS - Run #2; 3/3 job lulus

4.5 Hasil

Implementasi CI/CD ESORIA telah terbukti bekerja end-to-end pada GitHub Actions Run #2. Quality gate pengujian, publikasi image ke GHCR, dan pemasangan pada mesin Ubuntu WSL2 semuanya berhasil, dan verifikasi /api/release cocok dengan commit main. Mekanisme rollback berada dalam source serta tes regresi, tetapi belum dibuktikan melalui kegagalan nyata pada mesin produksi. Bukti eksperimen dan batasan tersebut dinyatakan secara eksplisit pada Lampiran D dan E.

BAB V
LAMPIRAN, KONTRIBUSI PENGERJAAN DAN KESIMPULAN

5.1 Lampiran UTS ESORIA

Berikut lampiran terkait UTS ESORIA:

1. Hasil video demo kami upload melalui YouTube. Video demo uts esoria: https://www.youtube.com/watch?v=zcOF-xwKlBs
2. Folder lengkap ESORIA: UTS - KELOMPOK ESORIA (PENGEMBANGAN DAN OPERASI)
3. Repository GITHUB: https://github.com/fikriianwarr/esoria-codefest-uts

5.2 Kontribusi Pengerjaan

Pembagian tugas Kelompok Esoria: Fikri Anwar (231112145) bertanggung jawab pada materi Docker dan Compose; Muhammad Alfisyahrin (231112853) pada GitHub Actions, GHCR, runner dan CD; Havidz Andrian (231111899) pada pengujian, pemeriksaan API/SQLite, dokumentasi bukti, dan laporan. Seluruhnya menjelaskan bagian berbeda dalam SATU VIDEO KELOMPOK. Tabel berikut merupakan pembagian kerja/demonstrasi yang disarankan; kontribusi historis perlu dikonfirmasi dari aktivitas dan rekaman sebenarnya.

Anggota (NIM)	Tugas Video Demonstrasi	Referensi Bukti untuk Video
Fikri Anwar 231112145	Koordinasi *containerization* dan aplikasi. Bertanggung jawab atas *Dockerfile* (*multi-stage build*), *Docker Compose* (*service web*, *volume*), *launcher*, konfigurasi keamanan *non-root*, *healthcheck* API, serta demonstrasi antarmuka website dan *runtime*.	Dockerfile; compose*.yaml; mulai.cmd; SS 04-08, 16, 18.
Havidz Andrian 231111899	Melaksanakan pengujian dan dokumentasi QA, termasuk pengujian lokal tahap kedua setelah pembaruan kode dengan hasil 94 tes lulus (85 Windows natif dan 9 Linux terisolasi), smoke test, audit persistensi SQLite, serta penyusunan laporan akhir.	tests/; verify-container*.mjs/sh; SS 09-10, 13, 17, 19; laporan Word.
Muhammad Alfisyahrin 231112853	Koordinasi *Pipeline CI/CD*. Mengelola alur *GitHub Actions* (*verify*, *publish*, *deploy*), integrasi *GHCR* untuk *image registry*, konfigurasi *self-hosted runner* (Linux/WSL2), serta implementasi skrip *deployment* dan *rollback*.	esoria-ci-cd.yml; scripts/deploy.sh; SS 11-15; Run #2.

Rekap rubrik: (1) container 50 poin didukung bukti build, port, non-root user, volume, uji persistensi, dan status healthy (SS 04-10, SS 16-17); (2) CI/CD 20 poin didukung Run #2 sukses, GHCR digest, runner, dan SHA rilis (SS 11-15, SS 17); (3) video 20 poin memerlukan SATU tautan video yang dapat diakses tanpa login, segmen container serta

CI/CD, dan penjelasan ketiga anggota; (4) laporan 10 poin memuat profil, kode, implementasi, evaluasi, dan SS 01-19. Nilai akhir ditetapkan dosen.

5.3 Kesimpulan

Berdasarkan hasil implementasi dan pengujian, proyek UTS pada aplikasi ESORIA CodeFest berhasil menerapkan teknologi container menggunakan Docker dan otomatisasi Continuous Integration/Continuous Deployment (CI/CD) melalui GitHub Actions. Aplikasi berbasis Node.js dan SQLite dapat dijalankan menggunakan Docker Compose dengan konfigurasi yang terstruktur, penyimpanan data persisten, serta pemeriksaan kesehatan layanan. Pada tahap awal, sebanyak 81 pengujian berhasil dijalankan melalui GitHub Actions Run #2. Setelah pembaruan dan penyempurnaan kode, pengujian lokal versi final menghasilkan 94 pengujian berhasil tanpa kegagalan. Hasil tersebut menunjukkan adanya perluasan cakupan pengujian pada tahap akhir pengembangan aplikasi.

Implementasi pipeline CI/CD juga berhasil menjalankan tiga tahapan utama, yaitu verify, publish, dan deploy, melalui GitHub Actions Run #2 pada 27 Oktober 2026 dengan status Success. Image aplikasi berhasil dipublikasikan ke GitHub Container Registry (GHCR) dan diterapkan melalui self-hosted runner Ubuntu WSL2. Proses deployment selesai dengan total durasi workflow 9 menit 48 detik, disertai verifikasi kesehatan aplikasi dan kesesuaian identitas rilis.

Secara keseluruhan, proyek ini menunjukkan bahwa penerapan Docker dan CI/CD dapat membantu meningkatkan konsistensi lingkungan aplikasi, mengotomatisasi pengujian, serta mempermudah proses deployment. Melalui pembagian tugas dan pengujian yang telah dilakukan, kelompok memperoleh pemahaman praktis mengenai penerapan konsep DevOps dalam pengembangan aplikasi. Meskipun demikian, pengujian deployment otomatis setelah push baru dan simulasi rollback secara langsung masih menjadi bagian yang dapat dikembangkan pada tahap selanjutnya.

DAFTAR PUSTAKA

1. Kelompok ESORIA CodeFest, “ESORIA CodeFest UTS: Source Code dan Dokumentasi Implementasi Container serta CI/CD,” *GitHub Repository*. https://github.com/fikriianwarr/esoria-codefest-uts
2. Kelompok ESORIA CodeFest, “ESORIA UTS - CI and CD: Konfigurasi Workflow GitHub Actions (esoria-ci-cd.yml),” *GitHub Repository*. https://github.com/fikriianwarr/esoria-codefest-uts/blob/main/.github/workflows/esoria-ci-cd.yml
3. GitHub Actions, “ESORIA UTS - CI and CD: Workflow Run #2 (Success),” *GitHub*. https://github.com/fikriianwarr/esoria-codefest-uts/actions/runs/40434638852
4. Video Demo dan Penjelasan UTS Esoria Codefest https://youtu.be/zcOF-xwKlBs?si=B2GS8hU2wZ_ZyFdP
5. Docker Inc., “Dockerfile Reference,” *Docker Documentation*. https://docs.docker.com/reference/dockerfile/
6. Docker Inc., “Compose File Reference,” *Docker Documentation*. https://docs.docker.com/reference/compose-file/
7. GitHub, Inc., “Workflow Syntax for GitHub Actions,” *GitHub Documentation*. https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax
8. GitHub, Inc., “Working with the Container Registry,” *GitHub Documentation*. https://docs.github.com/en/packages/working-with-a-github-packages-registry/working-with-the-container-registry
9. GitHub, Inc., “Adding Self-Hosted Runners,” *GitHub Documentation*. https://docs.github.com/en/actions/how-tos/manage-runners/self-hosted-runners/add-runners
10. GitHub, Inc., “Using Secrets in GitHub Actions,” *GitHub Documentation*. https://docs.github.com/en/actions/how-tos/write-workflows/choose-what-workflows-do/use-secrets
11. OpenJS Foundation, “SQLite (node),” *Node.js v24 Documentation*. https://nodejs.org/download/release/latest-v24.x/docs/api/sqlite.html

LAMPIRAN

A. MATRIKS PEMENUHAN RUBRIK

Tabel berikut mengaitkan setiap komponen penilaian dengan implementasi yang kami gunakan dan bukti screenshot yang disertakan. Status dibatasi pada aspek yang benar-benar didukung oleh foto terminal, antarmuka, dan halaman workflow.

Komponen rubrik	Pelaksanaan yang terbukti	Bukti
Container - 50 poin	Docker multi-stage build, service healthy, port 8088, user non-root, named volume, 81 tes pada pengujian awal CI/CD, 94 tes pada pengujian lokal final, serta uji restart/recreate.	SS 04-10, SS 16-17
CI/CD - 20 poin	Repository, YAML, runner WSL2, backup database, Verify + Publish + Deploy sukses pada Run #2	SS 11-17
Video - 20 poin	Penjelasan demo telah dibagi untuk tiga anggota; rekaman berisi demo dan penjelasan terkait web esoria.	BAB V
Laporan - 10 poin	BAB I-V, memuat detail aplikasi, arsitektur website, hasil data uji, kontribusi kelompok, referensi, tabel evaluasi, lampiran bukti uji, dokumentasi visual.	Laporan ini dan Lampiran B-E

Batas pembuktian: proyek memiliki tiga kategori seed dan dashboard admin menampilkan nol akun peserta serta nol pendaftaran pada foto akhir. Hal ini menunjukkan halaman admin tampil, bukan bukti adanya transaksi peserta. Pengujian rollback aktual di mesin deployment dan pembuktian perubahan setelah push baru belum tersedia; yang ada adalah pengujian regresi pada test suite.

LAMPIRAN

B. REKAP PENGUJIAN OTOMATIS

Pengujian ESORIA CodeFest dilakukan dalam dua tahap. Pengujian pertama pada versi awal aplikasi berhasil menjalankan 81 dari 81 tes tanpa kegagalan, sebagaimana tercatat pada GitHub Actions Run #2 tanggal 27 Oktober 2026. Setelah dilakukan pembaruan dan penyempurnaan kode, kelompok melaksanakan pengujian lokal tahap kedua sebagai pengujian versi final. Hasil rekapitulasi menunjukkan bahwa 94 dari 94 pengujian berhasil diselesaikan tanpa kegagalan, terdiri atas 85 pengujian Windows natif dan 9 pengujian Linux terisolasi. Dengan demikian, jumlah pengujian yang terdokumentasi bertambah sebanyak 13 pada tahap final. Hasil kedua tahap tersebut disajikan secara terpisah untuk menunjukkan perkembangan proses pengujian aplikasi.

Indikator	Hasil tercatat
Test suites	0 (runner bawaan Node.js melaporkan suites 0)
Jumlah tes	94
Lulus	94
Gagal	0
Dilewati	0
Dibatalkan	0
Sumber bukti	*Gambar E.09. SS_09_Automated_Testing _81* dan *Gambar E.09a. SS_09a_Automated_Testing_Final_94*

Pengujian tambahan: build image berhasil, audit runtime membuktikan akses HTTP, volume dan persistensi melalui restart/recreate. Perintah dan bukti pendukung ditunjukkan pada SS 04-10.

LAMPIRAN

C. PROSEDUR DAN HASIL VERIFIKASI CONTAINER

Pada Windows, source berada di folder esoria-codefest-uts. Docker Desktop terhubung dengan Ubuntu WSL2. Contoh perintah reproduksi yang telah dibuktikan melalui screenshot:

docker compose ps

docker compose port web 3000

docker compose logs --tail 30 web

npm run verify:container:windows

docker ps --filter name=esoria-uts-web-1

curl -fsS http://127.0.0.1:8088/api/health

curl -fsS http://127.0.0.1:8088/api/release

docker volume inspect esoria_uts_data --format '{{.Name}}'

Hasil pengujian pascadeployment: container esoria-uts-web-1 berstatus Up (healthy), pemetaan 127.0.0.1:8088->3000/tcp, API health mengembalikan status=ok/database=ok, dan volume esoria_uts_data tetap ada. Rilis versi 1.0.0 memiliki SHA commit yang cocok dengan workflow Run #2 (SS 16-17).

Pencadangan SQLite dilakukan sebelum CD dengan SQLite Backup API. Pemeriksaan integrity_check=ok dan hash SHA256 salinan cocok tercatat pada SS 13. Data disimpan di luar repository; tidak dipublikasikan bersama laporan untuk menjaga privasi. Hindari docker compose down -v, karena opsi -v menghapus volume terkait.

LAMPIRAN

D. PROSEDUR DAN HASIL CI/CD

Alamat repository: https://github.com/fikriianwarr/esoria-codefest-uts

Alamat workflow: https://github.com/fikriianwarr/esoria-codefest-uts/blob/main/.github/workflows/esoria-ci-cd.yml

Bukti eksekusi: https://github.com/fikriianwarr/esoria-codefest-uts/actions/runs/40434638852

Tahapan	Status	Durasi Run #2
Verify: tests dan container	Success	1 menit 13 detik
Publish: image teruji ke GHCR	Success	29 detik
Deploy: Ubuntu WSL2 self-hosted	Success	1 menit 21 detik
Workflow keseluruhan, termasuk antrean	Success	9 menit 48 detik

Runner esoria-wsl-uts memakai label self-hosted, Linux, X64, esoria-uts. Runner sempat offline sehingga job Deploy mengantre, kemudian ./run.sh mengaktifkannya kembali; setelah pekerjaan selesai kembali Idle (SS 15).

Image GHCR (digest hasil publish): ghcr.io/fikriianwarr/esoria-codefest-uts/esoria-uts@sha256:98fc41ff98001539068478aadddc9bdb5661e5e03125fbfa66bf425199d8bfd9

SHA commit yang dijalankan: 5fe69f90675504bcfc599efebd9fd5de04e5bc98

Validasi release pada /api/release menunjukkan version=1.0.0, appName=ESORIA CodeFest, dan SHA yang sama. Run #2 dipicu manual melalui workflow_dispatch pada branch main. Tidak ada klaim telah menguji pemicu push baru; bukti terkait dapat ditambahkan jika benar-benar dilaksanakan.

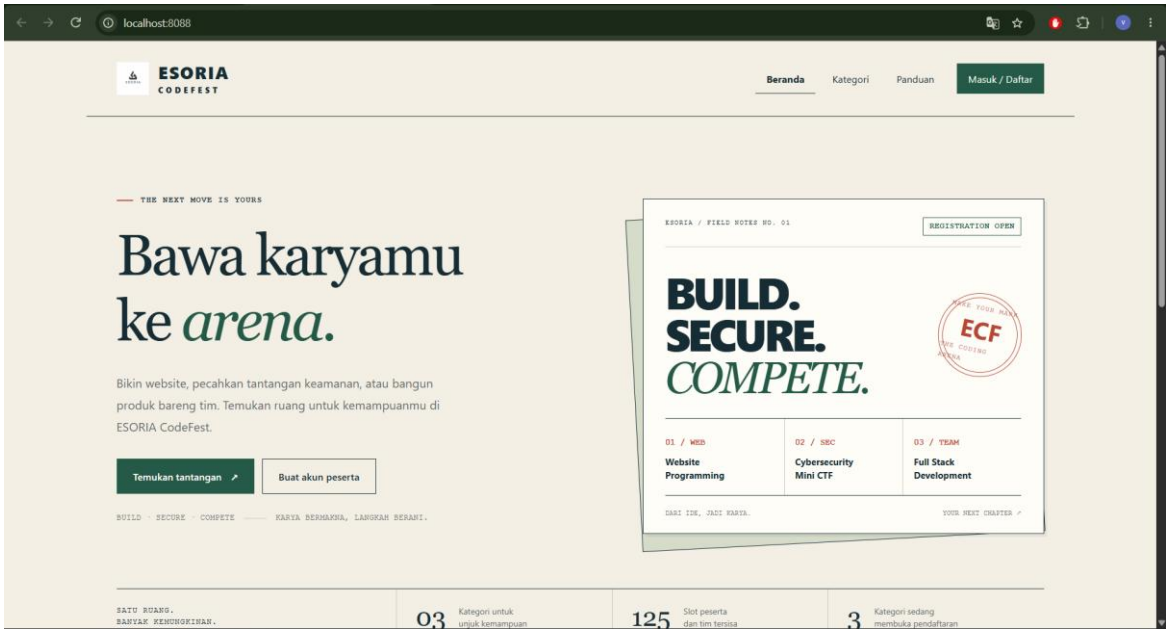
Keamanan: repository tetap Private dan hanya pengguna terotorisasi yang boleh menyalakan runner dengan akses Docker. Secrets admin tidak dicetak. Sebelum mengubah repository menjadi Public, runner lokal yang memiliki akses Docker perlu dilepas atau diamankan.

LAMPIRAN

E. DOKUMENTASI BUKTI VISUAL SS 01-19

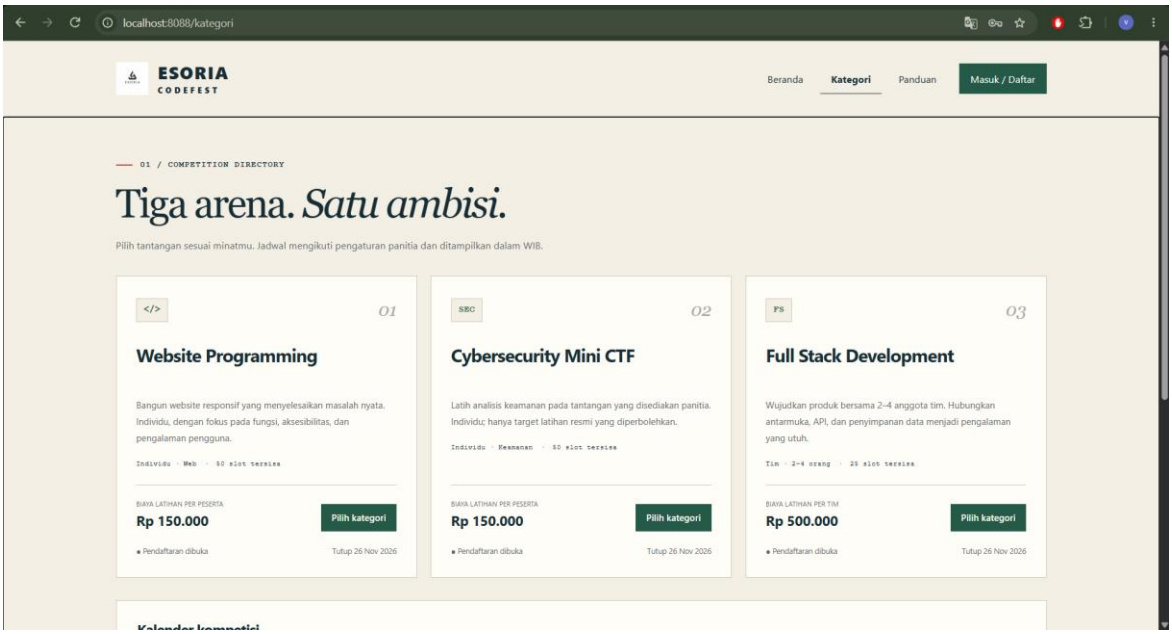
Lampiran ini memuat tangkapan layar asli pelaksanaan UTS pada komputer kelompok (27 Oktober 2026). Area kosong di bawah sebagian screenshot terminal dipangkas untuk memperjelas baris perintah dan hasil; isi teks bukti tidak diubah. Nama berkas dipertahankan sebagai penanda bukti.

SS 01 | Tampilan awal ESORIA CodeFest



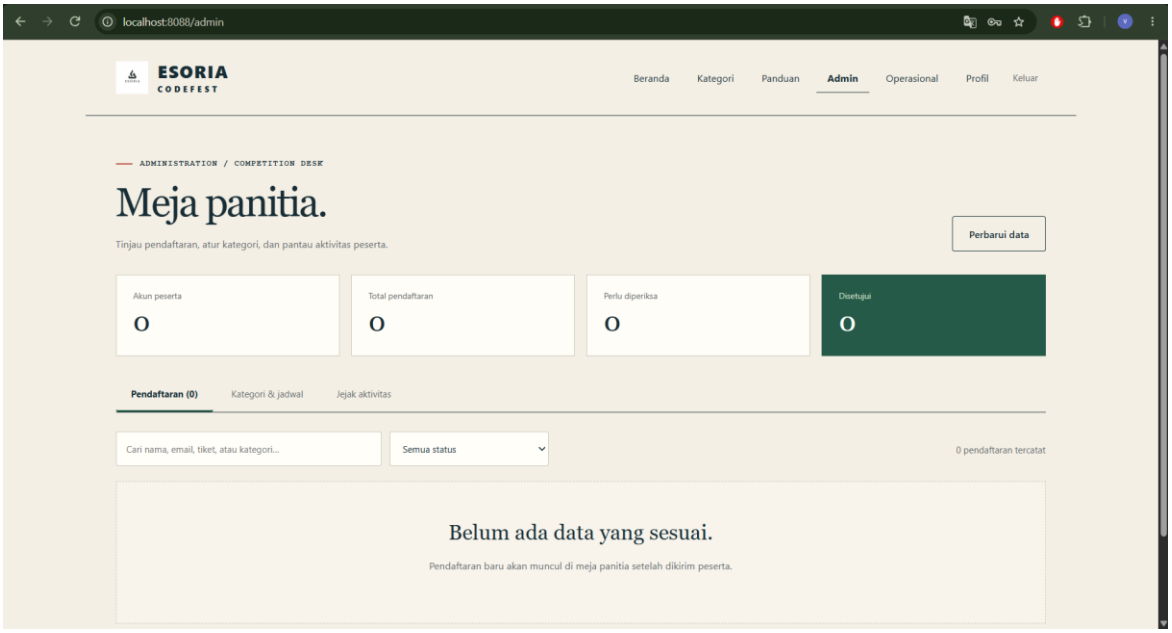
Gambar E.01. SS_01_Beranda_ESORIA

SS 02 | Tiga kategori kompetisi beserta kuota dan tarif simulasi



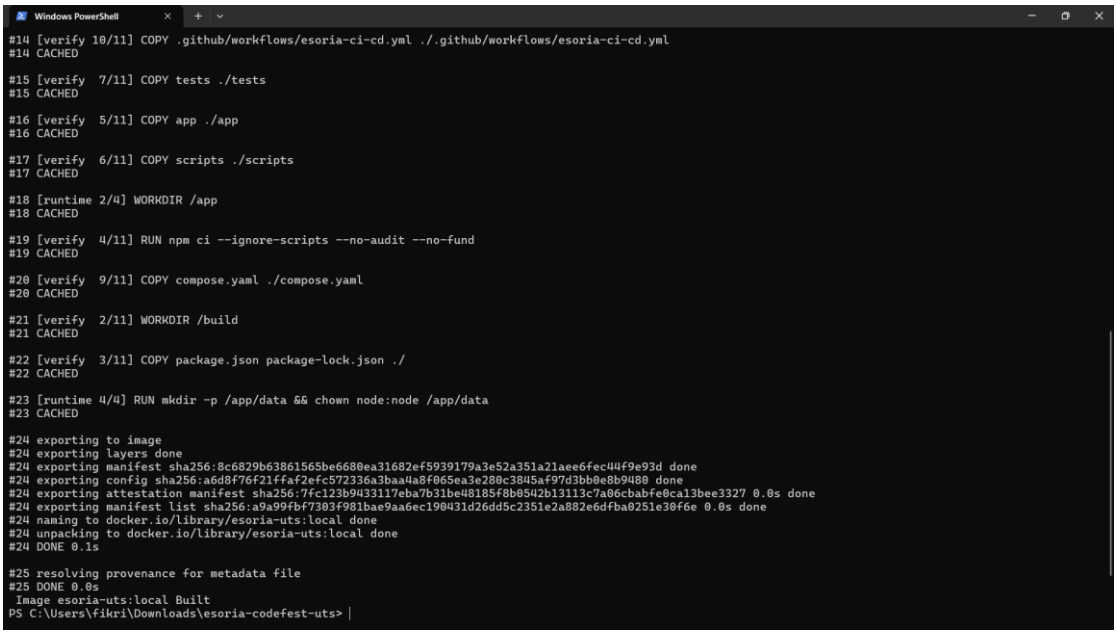
Gambar E.02. SS_02_Kategori_Lomba

SS 03 | Dashboard Admin dan kondisi data pada lingkungan uji



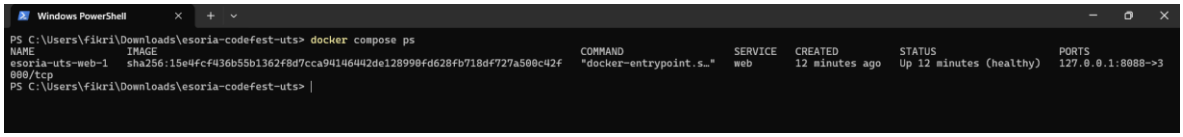
Gambar E.03. SS_03_Dashboard_Admin

SS 04 | Docker multi-stage build berhasil membentuk image lokal



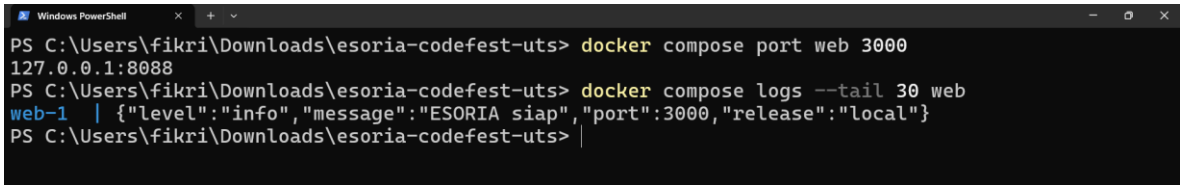
Gambar E.04. SS_04_Build_Docker_Berhasil

SS 05 | Service Docker berstatus healthy



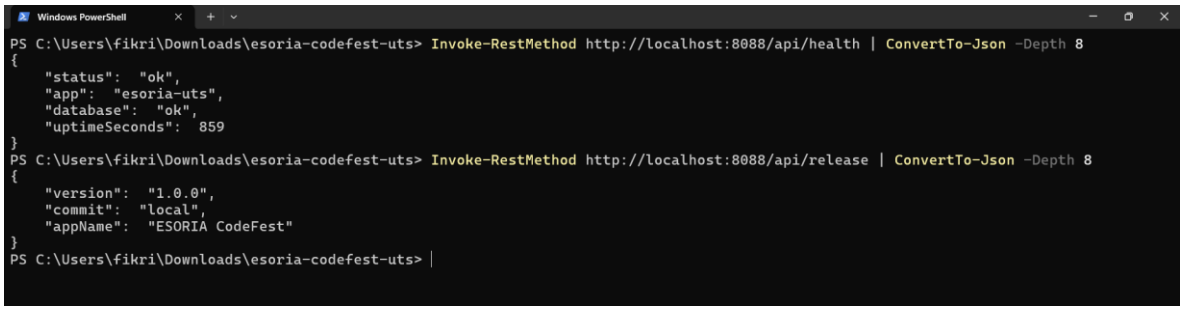
Gambar E.05. SS_05_Container_Healthy

SS 06 | Port 8088 dan startup log service web



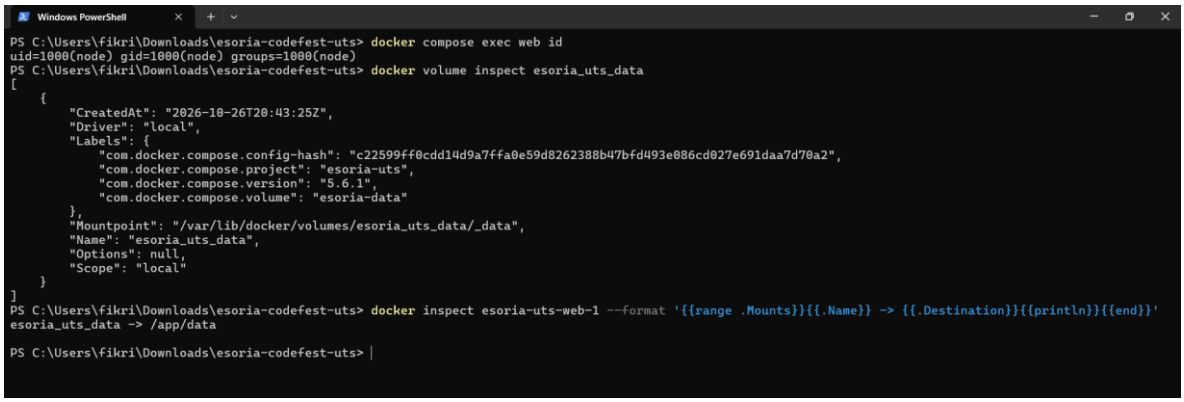
Gambar E.06. SS_06_Port_dan_Log_Docker

SS 07 | API health dan rilis lokal sebelum CI/CD



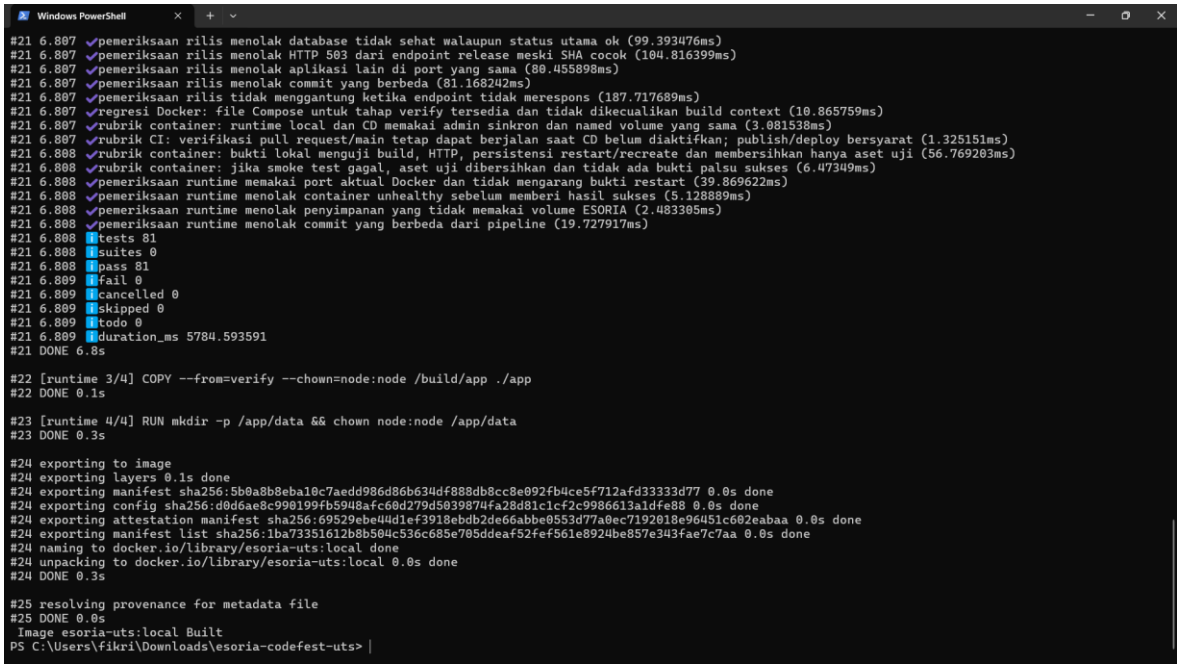
Gambar E.07. SS_07_Healthcheck_dan_Release

SS 08 | User node non-root dan volume database yang persisten

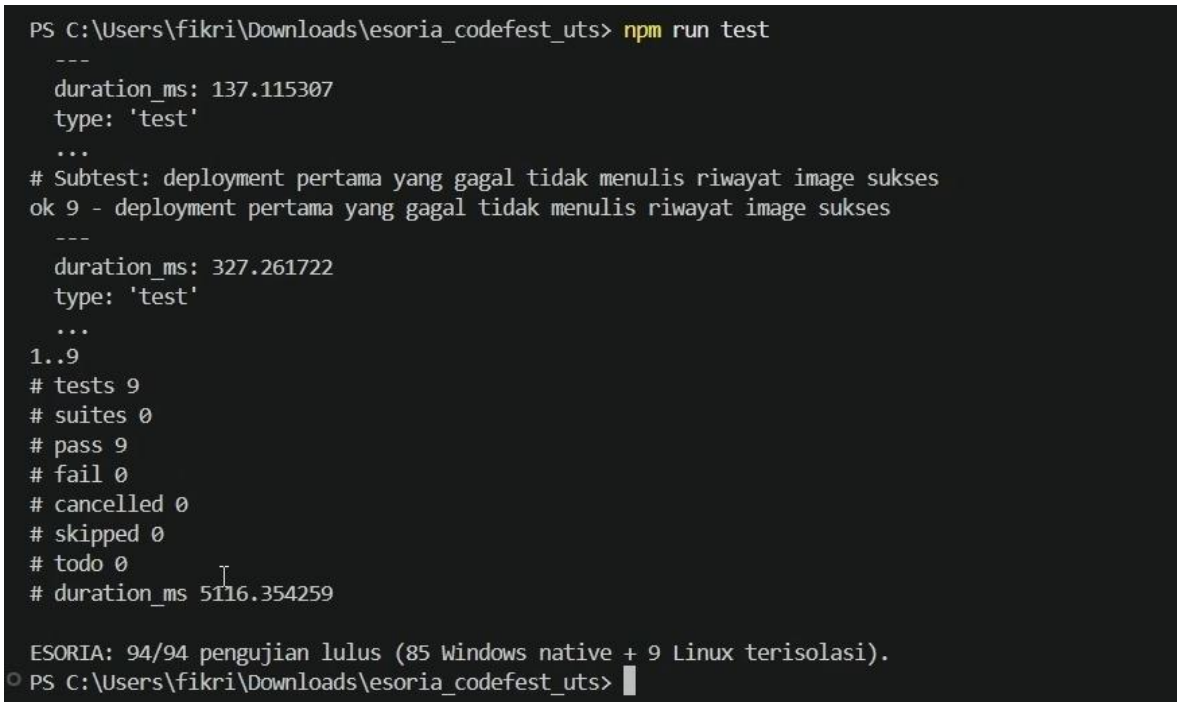


Gambar E.08. SS_08_Keamanan_dan_Volume_Database

SS 09 | Output pengujian: 81 lulus, 0 gagal

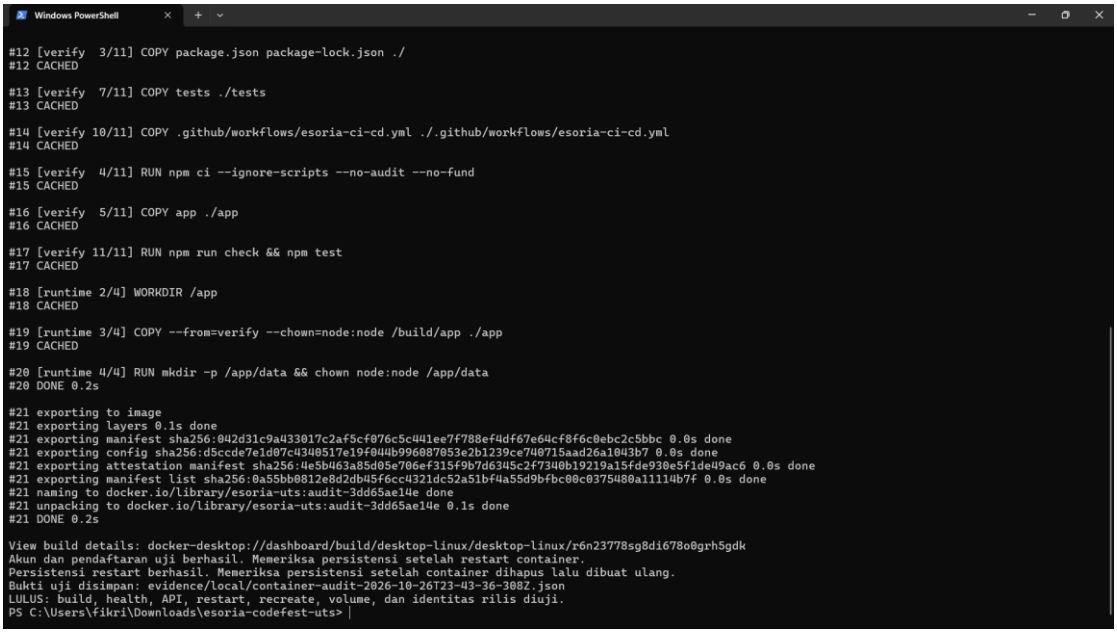


Gambar E.09. SS_09_Automated_Testing_Early _81



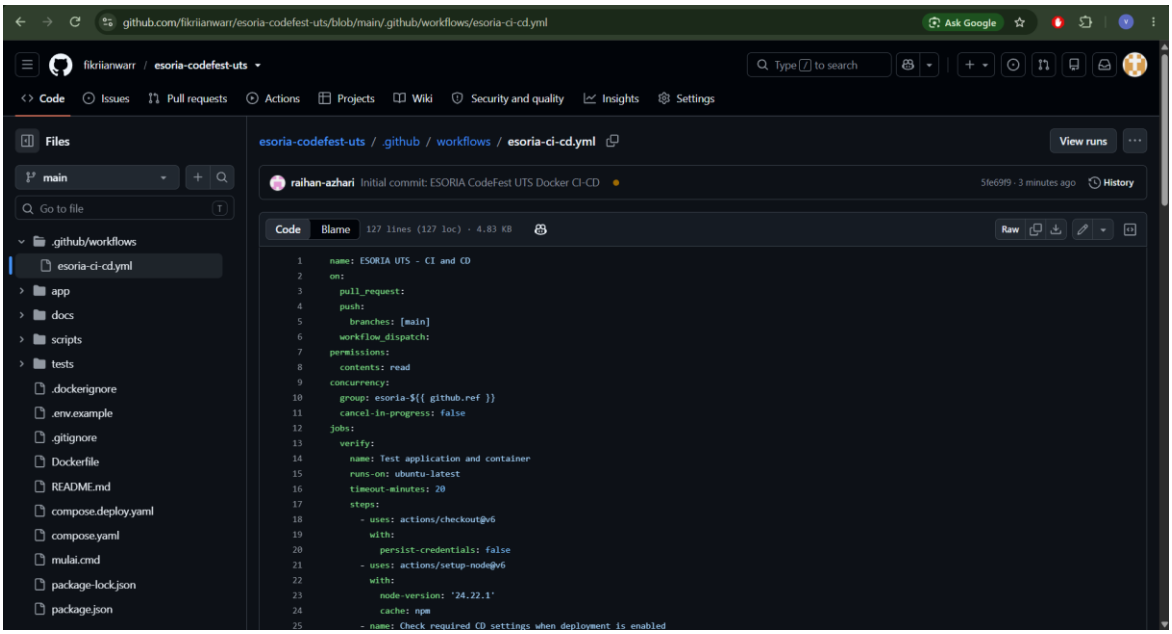
Gambar E.09a. SS_09a_Automated_Testing_Final_94

SS 10 | Audit restart/recreate dan persistensi container



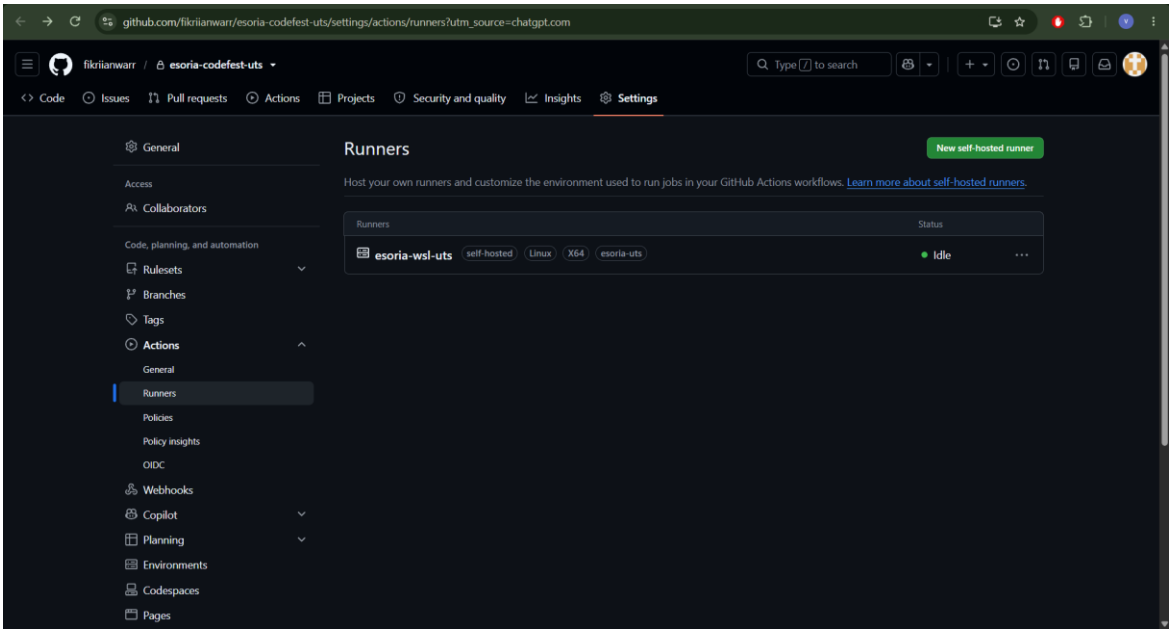
Gambar E.10. SS_10_Persistensi_Database_Docker

SS 11 | Repository GitHub dengan workflow esoria-ci-cd.yml



Gambar E.11. SS_11_Repository_Workflow_GitHub

SS 12 | Runner Ubuntu terdaftar dan status Idle



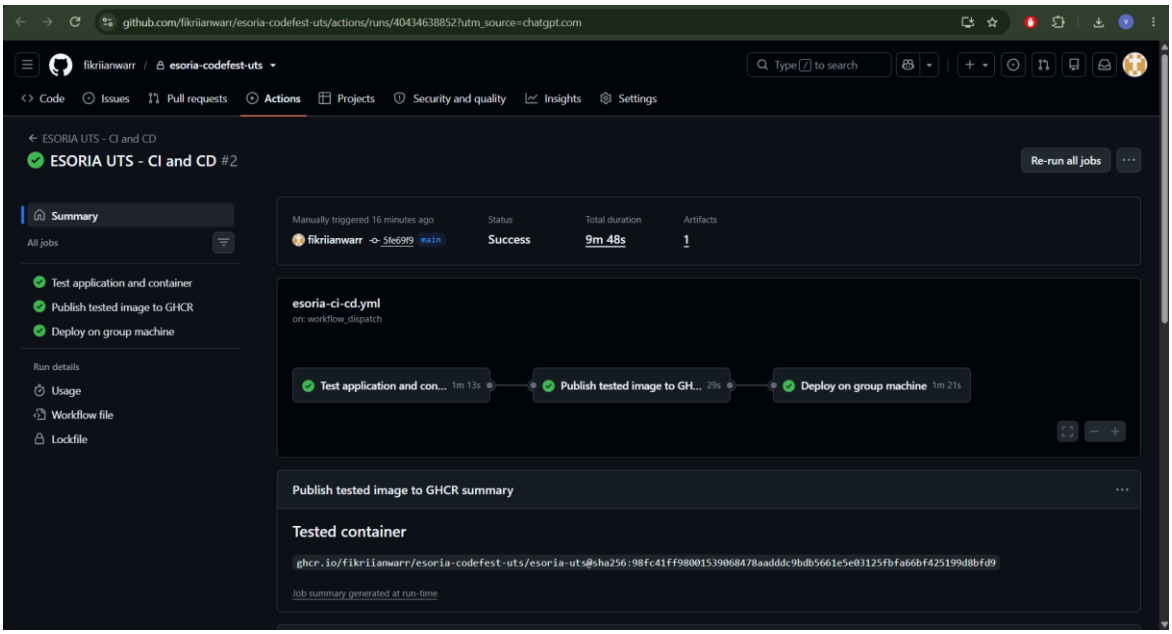
Gambar E.12. SS_12_Self_Hosted_Runner_Online.png - BAB IV - konfigurasi runner.

SS 13 | Backup SQLite valid dan runner memulai job deploy



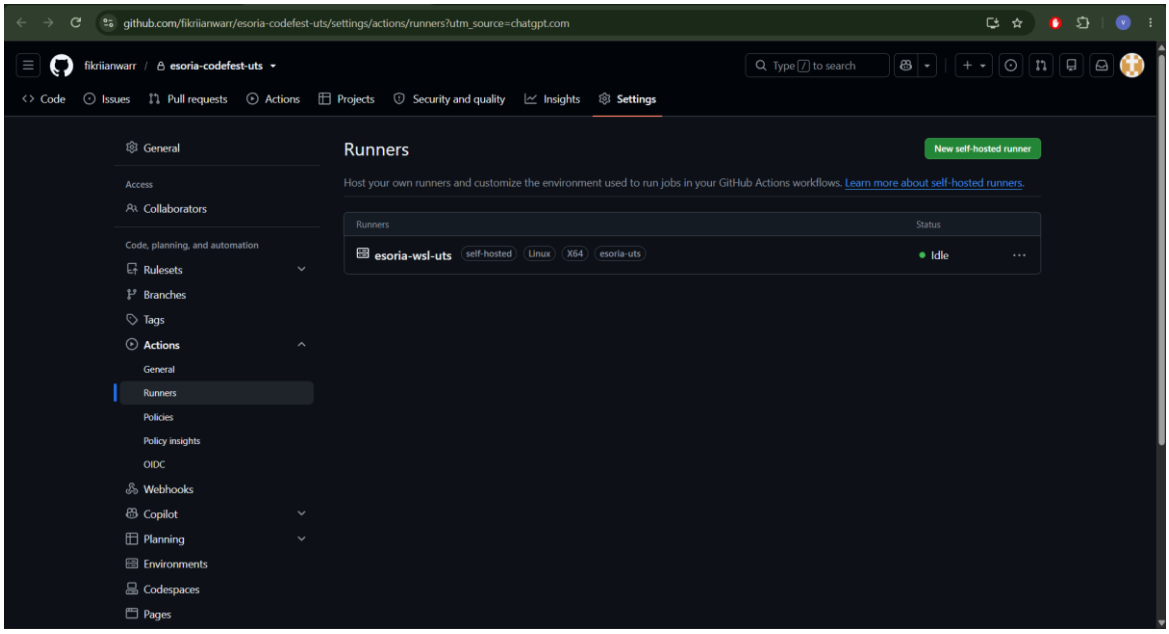
Gambar E.13. SS_13_Backup_Database_Dan_Runner_Deploy

SS 14 | Verify, Publish, Deploy berhasil dalam satu workflow



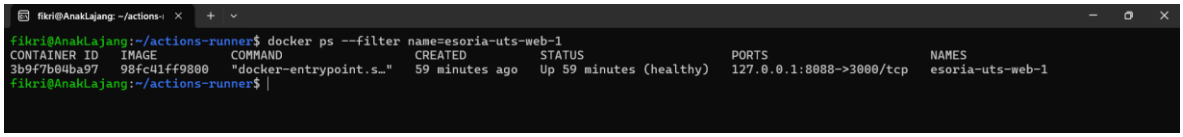
Gambar E.14. SS_14_CICD_Success

SS 15 | Runner esoria-wsl-uts kembali Idle setelah job



Gambar E.15. SS_15_Runner_Online

SS 16 | Container image hasil deployment dalam kondisi healthy



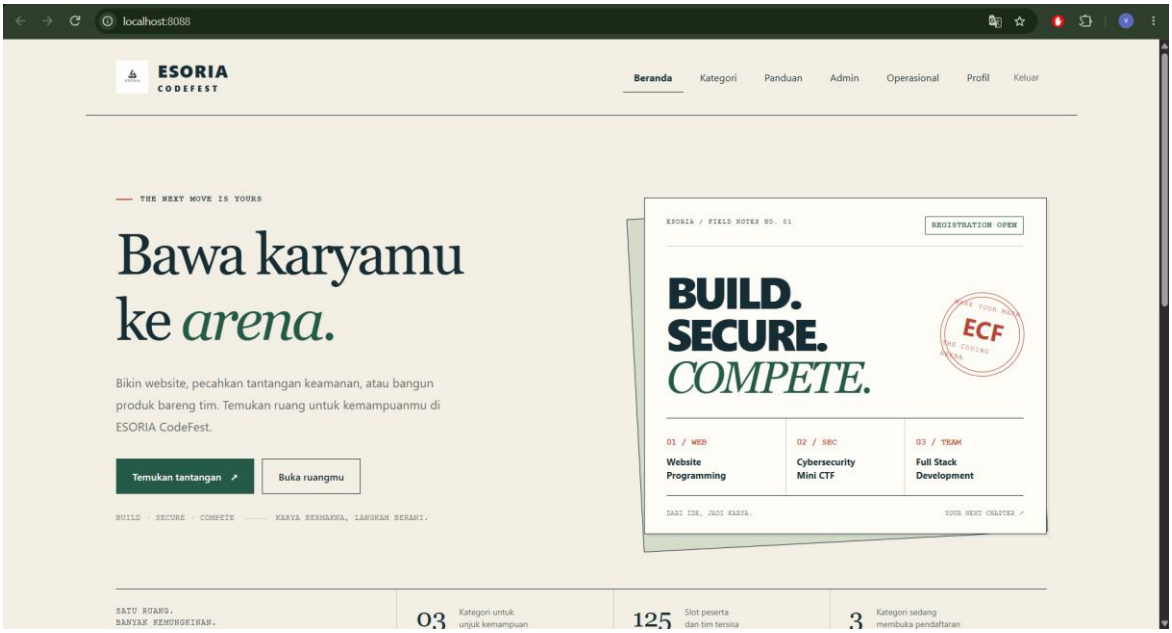
Gambar E.16. SS_16_Docker_Healthy

SS 17 | API health, identitas commit release, dan volume database



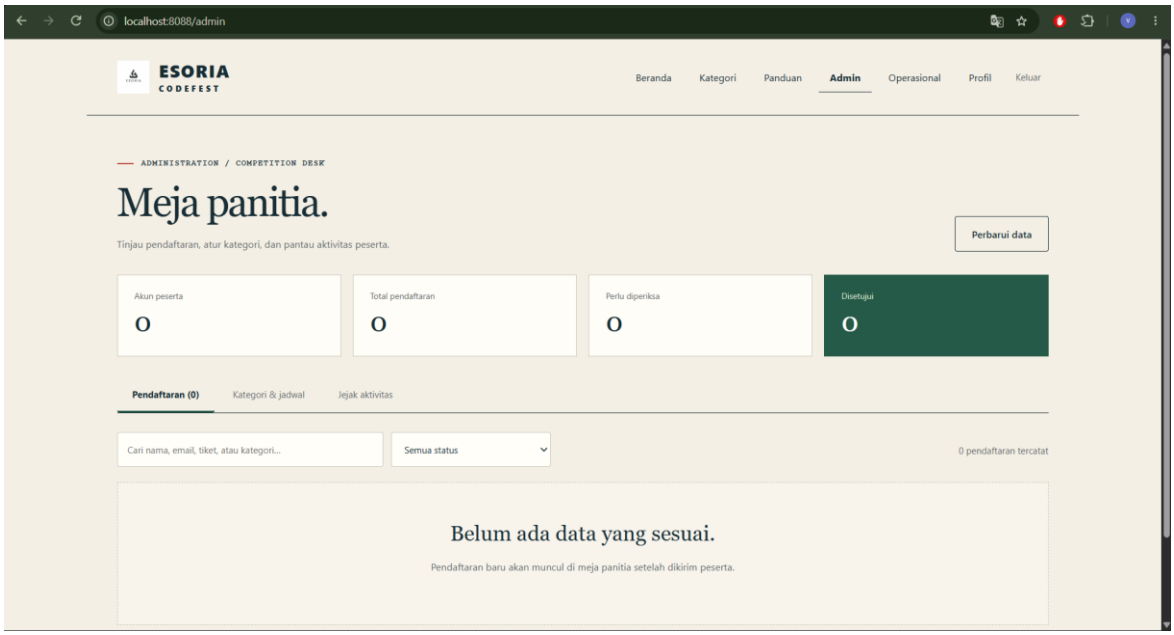
Gambar E.17. SS_17_Verifikasi_Deployment.

SS 18 | Beranda dapat diakses melalui localhost:8088 setelah deploy



Gambar E.18. SS_18_Website_Final

SS 19 | Halaman Admin dapat diakses setelah deploy; data pendaftaran masih nol



Gambar E.19. SS_19_Admin_Setelah_Deployment